Smart B2B Logistics & Delivery Optimization Platform

Complete Project Plan & Engineering Blueprint

Stack: MERN (MongoDB, Express.js, React.js, Node.js)

Classification: Enterprise-Grade Portfolio System

Team Size: 3 Developers

Timeline: 3-4 Weeks (28 Days)

Date: July 2026

MongoDB | Express.js | React.js | Node.js | Redux Toolkit | Mapbox | Tailwind CSS

Table of Contents

1. Executive Summary & Market Problem 3

2. User Ecosystem & Role Definitions 4

3. Complete Project Structure 5

4. Database Models (Full Schemas) 7

5. API Endpoints (Complete Reference) 11

6. Frontend Pages & Component Architecture 14

7. Real-Time Workflow (Step-by-Step) 17

8. Mobile Resiliency Engineering 19

9. MongoDB Aggregation Pipelines 21

10. Implementation Phases (Day-by-Day) 22

11. Team Task Division 26

12. Seed Data 28

13. Environment Configuration 29

14. Setup Commands 30

15. Engineering Interview Talking Points 31

1. Executive Summary & Market Problem

1.1 The Problem

In modern supply chains, B2B logistics providers face structural operational frictions that degrade margins and cause SLA violations. This platform solves three critical problems:

🔍 Black Box Visibility

Enterprise clients ordering high-value wholesale shipments are left in the dark regarding precision freight ETAs, causing administrative gridlock and heavy support overhead.

🚚 Poor Asset Utilization

Trucks operate below maximum load or follow redundant routes because dispatch planners lack immediate capacity visibility.

📄 Manifest Corruption

Fragmented order processing via static spreadsheets introduces data entry defects, dropped fields, and historical record misalignment.

🔗 No Real-Time Sync

Phone calls, text messages, and manual spreadsheets create delays and miscommunication across the entire supply chain.

1.2 The Solution

The platform unifies three critical transactional roles into a closed-loop system. By moving intensive data parsing to the database layer via MongoDB pipelines and decoupling frontend synchronization through cached wizards, the architecture achieves resilient execution ready for technical recruiters.

Core Capabilities:

- **Unified Backend Engine:** One server serves different frontend views based on user role
- **Real-Time Network Sync:** Any status update instantly reflects across all connected dashboards
- **Mobile Resilience:** Driver app survives phone locks, background tabs, and battery death via delta-time tracking
- **Algorithmic Routing:** Auto-match cargo weight/volume to compatible vehicles
- **Executive Analytics:** Complex MongoDB aggregation pipelines produce actionable business intelligence

2. User Ecosystem & Role Definitions

Role	Operational Mandate	Interface Type	Key Pages
Fleet Admin / Dispatcher	Asset allocation, capacity management, manifest creation, exception handling, driver assignment	Desktop-optimized	Dashboard, Fleet Monitor, Manifest Wizard, Live Map
Corporate Business Client	Bulk order placement, contract rate monitoring, cargo lifecycle tracking, invoice reconciliation	Responsive Desktop	Dashboard, Place Order, Track Shipment, Invoices
Field Driver / Courier	On-the-road execution, route adherence, instant progress confirmations, delivery proof	Mobile-first Touchscreen	Driver Dashboard, Active Delivery, Stop Terminal
Executive / Strategy Lead	Macro performance audits, fuel efficiency tracking, distribution hub performance analysis	Desktop Analytics	Executive Analytics (charts, widgets, KPIs)

Key Architecture Principle: Data updates executed by one user layer dynamically re-render dashboards across the remaining tiers in real time. When a driver taps "Delivery Complete," the admin map updates, the client tracker lights up, and the invoice is generated — all instantly.

3. Complete Project Structure

```
b2b-logistics/
│
├── backend/
│   ├── config/
│   │   ├── db.js                    # MongoDB connection with retry logic
│   │   ├── cors.js                  # CORS origin whitelist
│   │   └── env.js                   # Environment variable loader
│   │
│   ├── middleware/
│   │   ├── auth.js                  # JWT verification + user attach
│   │   ├── roleGate.js              # Role-based access control (admin, client,
driver, executive)
│   │   ├── errorHandler.js          # Centralized error catcher + rollback
│   │   └── validate.js              # express-validator chain wrapper
│   │
│   ├── models/
│   │   ├── User.js                  # All 4 roles in one model
│   │   ├── Vehicle.js               # Fleet asset registry
│   │   ├── Manifest.js              # Core shipment model (statusTimeline)
│   │   ├── Invoice.js               # Financial records
│   │   └── Notification.js          # In-app notifications
│   │
│   ├── routes/
│   │   ├── auth.routes.js           # login, register, me, refresh
│   │   ├── user.routes.js           # admin user management
│   │   ├── vehicle.routes.js        # fleet CRUD + availability
│   │   ├── manifest.routes.js       # shipment lifecycle
│   │   ├── invoice.routes.js        # billing
│   │   ├── analytics.routes.js      # executive aggregation queries
│   │   └── notification.routes.js   # user notifications
│   │
│   ├── controllers/
│   │   ├── auth.controller.js
│   │   ├── user.controller.js
│   │   ├── vehicle.controller.js
│   │   ├── manifest.controller.js
│   │   ├── invoice.controller.js
│   │   ├── analytics.controller.js
│   │   └── notification.controller.js
│   │
│   ├── services/
│   │   ├── routeCalculator.js       # Haversine distance + duration estimation
│   │   ├── capacityMatcher.js       # Auto-match cargo to compatible vehicles
│   │   ├── invoiceGenerator.js      # Build invoice line items from manifest
│   │   └── emailService.js          # Nodemailer wrapper
│   │
│   ├── cron/
│   │   └── overdueSweep.js          # Daily midnight check for overdue deliveries
│   │
│   ├── utils/
│   │   ├── ApiError.js              # Custom error class (statusCode, message)
│   │   ├── ApiResponse.js           # Standardized {success, data, message}
```

```
│   │   └── helpers.js                  # generateTrackingId, formatDate, etc.
│   │
│   ├── seeds/
│   │   ├── seed.js                     # Master seed runner
│   │   ├── users.seed.js               # Admin, clients, drivers, executives
│   │   ├── vehicles.seed.js            # Sample fleet
│   │   └── manifests.seed.js           # Sample shipments
│   │
│   ├── server.js                       # Express app entry point
│   ├── package.json
│   └── .env
│
├── frontend/
│   ├── public/
│   │   └── index.html
│   │
│   ├── src/
│   │   ├── main.jsx                    # React DOM render
│   │   ├── App.jsx                     # BrowserRouter + route definitions
│   │   │
│   │   ├── store/                      # Redux Toolkit
│   │   │   ├── store.js                # configureStore
│   │   │   ├── authSlice.js            # user, token, isAuthenticated
│   │   │   ├── manifestSlice.js        # selectedManifest, filters
│   │   │   ├── vehicleSlice.js         # fleet list, selected vehicle
│   │   │   └── uiSlice.js              # sidebarOpen, modalState, loading
│   │   │
│   │   ├── services/                   # API layer
│   │   │   ├── api.js                  # Axios instance + interceptors
│   │   │   ├── authApi.js              # login, register, getProfile
│   │   │   ├── manifestApi.js          # CRUD + assign + status
│   │   │   ├── vehicleApi.js           # CRUD + available + stats
│   │   │   ├── invoiceApi.js           # generate, list, pay
│   │   │   ├── analyticsApi.js         # all executive endpoints
│   │   │   └── notificationApi.js      # get, markRead, markAllRead
│   │   │
│   │   ├── hooks/
│   │   │   ├── useAuth.js              # Read auth state from Redux
│   │   │   ├── useRecovery.js          # Mobile: recover trip state from localStorage
│   │   │   ├── useLocalStorage.js      # Persist wizard form data
│   │   │   └── useDebounce.js          # Debounce search inputs
│   │   │
│   │   ├── components/
│   │   │   ├── layout/
│   │   │   │   ├── AppShell.jsx            # Main layout wrapper (sidebar + content)
│   │   │   │   ├── Sidebar.jsx             # Role-based navigation menu
│   │   │   │   ├── Topbar.jsx              # User info, notifications bell, logout
│   │   │   │   └── ProtectedRoute.jsx      # Auth guard + role redirect
│   │   │   │
│   │   │   ├── shared/
│   │   │   │   ├── StatusBadge.jsx         # Colored status pill
│   │   │   │   ├── DataTable.jsx           # Sortable, filterable table
│   │   │   │   ├── StatCard.jsx            # Dashboard metric card
│   │   │   │   ├── ProgressStepper.jsx     # Pending > Assigned > In-Transit >
Delivered
│   │   │   │   ├── ConfirmModal.jsx        # Reusable confirmation dialog
│   │   │   │   ├── LoadingSpinner.jsx      # Full-page / inline spinner
│   │   │   │   ├── EmptyState.jsx          # No data placeholder
```

```
│   │   │   │   └── SearchInput.jsx         # Debounced search bar
│   │   │   │
│   │   │   ├── admin/
│   │   │   │   ├── FleetGrid.jsx              # Vehicle data grid with status
│   │   │   │   ├── AddEditVehicleModal.jsx     # Vehicle create/edit form
│   │   │   │   ├── ManifestWizard/
│   │   │   │   │   ├── WizardContainer.jsx     # Step manager + localStorage
│   │   │   │   │   ├── StepPartner.jsx         # Client select + origin/destination
│   │   │   │   │   ├── StepCargo.jsx           # Weight, volume, items, hazardous
│   │   │   │   │   └── StepRoute.jsx           # Auto-suggested vehicles + confirm
│   │   │   │   ├── LiveMap.jsx                 # Mapbox GL with vehicle markers
│   │   │   │   ├── DispatchPanel.jsx           # Assign driver + vehicle to manifest
│   │   │   │   └── ManifestDetailModal.jsx     # Full manifest view + timeline
│   │   │   │
│   │   │   ├── client/
│   │   │   │   ├── ClientOverview.jsx          # Stats: outstanding, spent, fulfillment
│   │   │   │   ├── OrderForm.jsx               # Bulk freight request form
│   │   │   │   ├── ShipmentTracker.jsx         # Progress stepper + timeline
│   │   │   │   └── InvoiceList.jsx             # Invoice table with status
│   │   │   │
│   │   │   ├── driver/
│   │   │   │   ├── RunSheet.jsx                # Daily delivery checklist
│   │   │   │   ├── StopTerminal.jsx            # Warehouse info + action buttons
│   │   │   │   ├── TripTimer.jsx               # Delta-time elapsed display
│   │   │   │   └── DeliveryConfirm.jsx         # Proof of delivery + complete
│   │   │   │
│   │   │   └── executive/
│   │   │       ├── FleetUtilizationChart.jsx   # Pie/bar: Available vs In-Transit vs
Maintenance
│   │   │       ├── RouteEfficiencyChart.jsx    # Line chart: avg distance vs actual
│   │   │       ├── MonthlyCapacityWidget.jsx   # Bar chart: monthly throughput
│   │   │       ├── DeliveryPerformance.jsx     # On-time % vs delayed % gauge
│   │   │       └── RevenueSummary.jsx          # Monthly revenue bar + total
│   │   │
│   │   ├── pages/
│   │   │   ├── Login.jsx                       # Unified role-based login gate
│   │   │   ├── NotFound.jsx                    # 404 page
│   │   │   ├── admin/
│   │   │   │   ├── AdminDashboard.jsx          # Summary cards + recent manifests
│   │   │   │   ├── FleetMonitor.jsx            # Full fleet grid page
│   │   │   │   ├── ManifestCreate.jsx          # Multi-step wizard page
│   │   │   │   └── LiveOperations.jsx          # Map + dispatch panel page
│   │   │   ├── client/
│   │   │   │   ├── ClientDashboard.jsx         # Client overview + recent orders
│   │   │   │   ├── PlaceOrder.jsx              # Order form page
│   │   │   │   ├── TrackShipment.jsx           # Single shipment tracking
│   │   │   │   └── ClientInvoices.jsx          # Invoice list page
│   │   │   ├── driver/
│   │   │   │   ├── DriverDashboard.jsx         # Today's deliveries list
│   │   │   │   └── ActiveDelivery.jsx          # Run-sheet + stop terminal + timer
│   │   │   └── executive/
│   │   │       └── ExecutiveAnalytics.jsx      # All charts + KPI widgets
│   │   │
│   │   ├── utils/
│   │   │   ├── constants.js                    # Status enums, role names, routes
│   │   │   ├── formatters.js                   # formatDate, formatCurrency,
formatWeight
│   │   │   └── validators.js                   # Email, phone, required field
```

```
validators
│   │   │
│   │   └── styles/
│   │       └── globals.css                    # Tailwind imports + custom styles
│   │
│   ├── index.html
│   ├── package.json
│   ├── vite.config.js
│   ├── tailwind.config.js
│   └── postcss.config.js
│
├── .gitignore
├── README.md
└── package.json                  # Root package.json (concurrently scripts)
```

4. Database Models (Full Schemas)

4.1 User Model

Single model for all 4 roles. The `role` field determines access level. Passwords are hashed with bcrypt on save.

```
const UserSchema = new mongoose.Schema(
  {
    firstName:     { type: String, required: true, trim: true },
    lastName:      { type: String, required: true, trim: true },
    email:         { type: String, required: true, unique: true, lowercase: true, trim:
true },
    password:      { type: String, required: true, select: false },  // Hidden by
default
    role:          { type: String, enum: ['admin', 'client', 'driver', 'executive'],
required: true },
    phone:         { type: String, default: '' },
    company:       { type: String, default: '' },        // Client-specific
    licenseNumber: { type: String, default: null },      // Driver-specific
    contractRate:  { type: Number, default: 0 },         // Client-specific (per km
rate)
    isActive:      { type: Boolean, default: true },
  },
  { timestamps: true }
);

// Auto-hash password before save
UserSchema.pre('save', async function (next) {
  if (!this.isModified('password')) return next();
  this.password = await bcrypt.hash(this.password, 12);
  next();
});

// Instance method for login comparison
UserSchema.methods.comparePassword = async function (candidatePassword) {
  return await bcrypt.compare(candidatePassword, this.password);
};

// Performance indexes
UserSchema.index({ email: 1 });
UserSchema.index({ role: 1 });
UserSchema.index({ role: 1, isActive: 1 });
```

Field Breakdown:

Field	Type	Required	Description
firstName	String	Yes	User's first name

lastName	String	Yes	User's last name
email	String	Yes	Unique login identifier, lowercase
password	String	Yes	bcrypt hashed, select:false (hidden in queries)
role	Enum	Yes	admin | client | driver | executive
phone	String	No	Contact number
company	String	No	Client's company name
licenseNumber	String	No	Driver's license ID
contractRate	Number	No	Client's per-km billing rate
isActive	Boolean	No	Soft-delete flag

4.2 Vehicle Model

```
const VehicleSchema = new mongoose.Schema(
  {
    registrationNumber:     { type: String, required: true, unique: true, trim: true },
    model:                  { type: String, required: true },
    make:                   { type: String, required: true },
    year:                   { type: Number, required: true },
    maxWeightKg:            { type: Number, required: true },     // Max payload
capacity
    maxVolumeCubicMeters:   { type: Number, required: true },     // Max cargo volume
    status: {
      type: String,
      enum: ['Available', 'In-Transit', 'Maintenance'],
      default: 'Available',
    },
    currentDriver:          { type: mongoose.Schema.Types.ObjectId, ref: 'User',
default: null },
    fuelEfficiencyKmPerLiter: { type: Number, default: 0 },
    lastMaintenanceDate:    { type: Date, default: null },
  },
  { timestamps: true }
);

VehicleSchema.index({ status: 1 });
VehicleSchema.index({ registrationNumber: 1 });
```

Field Breakdown:

Field	Type	Required	Description
registrationNumber	String	Yes	Unique plate number (e.g., TRK-001)
model	String	Yes	Truck model name

make	String	Yes	Manufacturer (Volvo, Mercedes, etc.)
year	Number	Yes	Manufacture year
maxWeightKg	Number	Yes	Maximum payload in kilograms
maxVolumeCubicMeters	Number	Yes	Maximum cargo volume in m^3
status	Enum	Yes	Available | In-Transit | Maintenance
currentDriver	ObjectId	No	Reference to assigned User (driver)
fuelEfficiencyKmPerLiter	Number	No	Fuel efficiency metric for analytics
lastMaintenanceDate	Date	No	Last service date for maintenance scheduling

4.3 Manifest Model (Core Engine)

This is the central model of the entire platform. Every shipment flows through this model's lifecycle.

```
const ManifestSchema = new mongoose.Schema(
  {
    trackingId: {
      type: String, required: true, unique: true, trim: true,
      default: () => `TRK-${Math.floor(100000 + Math.random() * 900000)}`,
    },
    client:   { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
    driver:   { type: mongoose.Schema.Types.ObjectId, ref: 'User', default: null },
    vehicle:  { type: mongoose.Schema.Types.ObjectId, ref: 'Vehicle', default: null },

    cargoDetails: {
      description:            { type: String, required: true },
      totalWeightKg:          { type: Number, required: true },
      totalVolumeCubicMeters: { type: Number, required: true },
      itemCount:              { type: Number, required: true },
      isHazardous:            { type: Boolean, default: false },
    },

    routing: {
      origin: {
        address:     { type: String, required: true },
        coordinates: { type: [Number], required: true },   // [longitude, latitude]
      },
      destination: {
        address:     { type: String, required: true },
        coordinates: { type: [Number], required: true },
      },
      estimatedDistanceKm:      { type: Number },
      estimatedDurationMinutes: { type: Number },
    },

    currentStatus: {
      type: String,
      enum: ['Pending', 'Assigned', 'In-Transit', 'Delivered', 'Delayed', 'Cancelled'],
```

```
      default: 'Pending',
    },

    statusTimeline: [
      {
        status:    { type: String, enum: ['Pending','Assigned','In-
Transit','Delivered','Delayed','Cancelled'], required: true },
        timestamp: { type: Date, default: Date.now },
        note:      { type: String, default: '' },
        updatedBy: { type: mongoose.Schema.Types.ObjectId, ref: 'User' },
      },
    ],

    scheduledPickup:               { type: Date, required: true },
    scheduledDeliveryWindowClose:  { type: Date, required: true },
    actualDeliveryTime:            { type: Date },

    // Mobile resilience fields
    tripStartTime:      { type: Date, default: null },
    tripStartTimestamp:  { type: Number, default: null },  // UNIX ms for delta-time
recovery
  },
  { timestamps: true }
);

// High-Performance Query Optimization Indexes
ManifestSchema.index({ trackingId: 1 });
ManifestSchema.index({ currentStatus: 1 });
ManifestSchema.index({ client: 1, createdAt: -1 });
ManifestSchema.index({ driver: 1, currentStatus: 1 });
ManifestSchema.index({ scheduledDeliveryWindowClose: 1, currentStatus: 1 });
```

Manifest Status Lifecycle:

Pending → **Assigned** → **In-Transit** → **Delivered**

Alternate paths: → Delayed (from any active state) | → Cancelled (from Pending or Assigned)

Field Breakdown:

Field	Type	Description
trackingId	String	Auto-generated unique ID (TRK-XXXXXX)
client	ObjectId → User	The corporate client who placed the order
driver	ObjectId → User	Assigned driver (null until assigned)
vehicle	ObjectId → Vehicle	Assigned truck (null until assigned)
cargoDetails.description	String	What is being shipped

cargoDetails.totalWeightKg	Number	Total weight in kilograms
cargoDetails.totalVolumeCubicMeters	Number	Total volume in cubic meters
cargoDetails.itemCount	Number	Number of individual items
cargoDetails.isHazardous	Boolean	Hazardous materials flag
routing.origin	Object	Pickup address + [lng, lat] coordinates
routing.destination	Object	Delivery address + [lng, lat] coordinates
routing.estimatedDistanceKm	Number	Calculated route distance
routing.estimatedDurationMinutes	Number	Estimated transit time
currentStatus	Enum	Current state in the lifecycle
statusTimeline[]	Array	Full audit trail of every status change
scheduledPickup	Date	When the shipment should be picked up
scheduledDeliveryWindowClose	Date	Deadline for delivery
actualDeliveryTime	Date	When delivery was actually completed
tripStartTimestamp	Number	UNIX ms timestamp for delta-time mobile recovery

4.4 Invoice Model

```
const InvoiceSchema = new mongoose.Schema(
  {
    invoiceNumber: {
      type: String, required: true, unique: true,
      default: () => `INV-${Date.now()}-${Math.floor(1000 + Math.random() * 9000)}`,
    },
    manifest:  { type: mongoose.Schema.Types.ObjectId, ref: 'Manifest', required: true
},
    client:    { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
    amount:    { type: Number, required: true },
    currency:  { type: String, default: 'USD' },
    status:    { type: String, enum: ['Pending', 'Paid', 'Overdue', 'Cancelled'],
default: 'Pending' },
    issuedDate: { type: Date, default: Date.now },
    dueDate:   { type: Date, required: true },
    paidDate:  { type: Date, default: null },
    lineItems: [
      {
        description: String,
        quantity:    Number,
        unitPrice:   Number,
        total:       Number,
      },
```

```
    ],
  },
  { timestamps: true }
);

InvoiceSchema.index({ client: 1, status: 1 });
InvoiceSchema.index({ invoiceNumber: 1 });
```

4.5 Notification Model

```
const NotificationSchema = new mongoose.Schema(
  {
    recipient:        { type: mongoose.Schema.Types.ObjectId, ref: 'User', required:
true },
    title:            { type: String, required: true },
    message:          { type: String, required: true },
    type:             { type: String, enum: ['info', 'warning', 'success', 'error'],
default: 'info' },
    isRead:           { type: Boolean, default: false },
    relatedManifest:  { type: mongoose.Schema.Types.ObjectId, ref: 'Manifest', default:
null },
  },
  { timestamps: true }
);

NotificationSchema.index({ recipient: 1, isRead: 1 });
```

5. API Endpoints (Complete Reference)

5.1 Auth Routes — `/api/auth`

Method	Endpoint	Access	Request Body	Response
POST	/register	Public	{firstName, lastName, email, password, role, company?, phone?}	{token, user}
POST	/login	Public	{email, password}	{token, user}
GET	/me	Authenticated	—	{user}
POST	/refresh	Authenticated	—	{token}

5.2 User Routes — `/api/users`

Method	Endpoint	Access	Description
GET	/	Admin	List all users with pagination & role filter
GET	/drivers	Admin	List all users with role=driver
GET	/clients	Admin	List all users with role=client
GET	/:id	Admin	Get single user details
PUT	/:id	Admin	Update user profile
PATCH	/:id/deactivate	Admin	Soft-delete (isActive=false)

5.3 Vehicle Routes — `/api/vehicles`

Method	Endpoint	Access	Description
GET	/	Admin/Executive	List all vehicles with status filter
GET	/available	Admin	List vehicles with status=Available
GET	/stats	Executive	Fleet utilization aggregated stats
POST	/	Admin	Add new vehicle
PUT	/:id	Admin	Update vehicle details

PATCH	/:id/status	Admin	Update vehicle status
DELETE	/:id	Admin	Remove vehicle from fleet

5.4 Manifest Routes — `/api/manifests`

Method	Endpoint	Access	Description
GET	/	Admin	List all manifests (filterable by status, date)
GET	/my	Client	List current client's manifests
GET	/driver/my	Driver	List driver's assigned active manifests
GET	/:id	All Roles	Get manifest full details + timeline
POST	/	Admin/Client	Create new manifest (auto-generates trackingId)
PUT	/:id	Admin	Update manifest details
PATCH	/:id/assign	Admin	Assign driver + vehicle, set status to Assigned
PATCH	/:id/start-trip	Driver	Start trip, capture UNIX timestamp, set In-Transit
PATCH	/:id/status	Driver	Update status, push to statusTimeline
PATCH	/:id/complete	Driver	Mark Delivered, set actualDeliveryTime
DELETE	/:id	Admin	Cancel manifest

5.5 Invoice Routes — `/api/invoices`

Method	Endpoint	Access	Description
GET	/	Admin	List all invoices
GET	/my	Client	List current client's invoices
GET	/:id	Admin/Client	Get invoice details
POST	/generate/:manifestId	System	Auto-generate invoice when delivery completes
PATCH	/:id/pay	Admin	Mark invoice as Paid
GET	/stats	Executive	Revenue summary aggregated data

5.6 Analytics Routes — `/api/analytics`

Method	Endpoint	Access	Description

GET	/fleet-utilization	Admin/Executive	% of fleet Available vs In-Transit vs Maintenance
GET	/route-efficiency	Admin/Executive	Estimated vs actual distance/duration
GET	/monthly-capacity	Executive	Monthly cargo throughput (kg & m^3)
GET	/delivery-performance	Executive	On-time % vs delayed % vs cancelled %
GET	/revenue-summary	Executive	Monthly revenue totals + pending amounts

5.7 Notification Routes — `/api/notifications`

Method	Endpoint	Access	Description
GET	/	Authenticated	Get current user's notifications (newest first)
PATCH	/:id/read	Authenticated	Mark single notification as read
PATCH	/read-all	Authenticated	Mark all user's notifications as read
GET	/unread-count	Authenticated	Get count of unread notifications

6. Frontend Pages & Component Architecture

6.1 Login Page (Unified Gate)

Single entry point for all 4 roles. Email + password form. On submit, backend returns JWT with role claims. React reads token and redirects to role-specific dashboard.

- **Admin** → `/admin/dashboard`
- **Client** → `/client/dashboard`
- **Driver** → `/driver/dashboard`
- **Executive** → `/executive/analytics`

6.2 Global Layout (AppShell)

Wraps all authenticated pages. Contains:

- **Sidebar:** Role-based navigation links. Admin sees fleet/manifest/map. Client sees orders/tracking/invoices. Driver sees deliveries. Executive sees analytics.
- **Topbar:** User name, notification bell (unread count badge), logout button.
- **Content Area:** React Router outlet for page content.

6.3 Admin Dashboard Page

4 stat cards at top: Total Manifests, Active Vehicles, Pending Orders, Overdue Deliveries. Below: recent manifests table with status badges and quick action buttons.

6.4 Fleet Monitor Page

Data grid showing all vehicles. Columns: Registration, Model/Make, Max Weight, Max Volume, Status (colored badge), Current Driver. Actions: Add Vehicle button opens modal, Edit/Delete on each row.

6.5 Manifest Create (Multi-Step Wizard)

3-step progressive form with localStorage persistence:

Step 1: Partner & Node Setup

- Select client from dropdown (populated from /api/users/clients)
- Enter origin address with geocoded coordinates
- Enter destination address with geocoded coordinates
- Set scheduled pickup date
- Set delivery window close date

Step 2: Cargo Volume Specifications

- Cargo description text field
- Total weight (kg) number input
- Total volume (m^3) number input
- Item count number input
- Hazardous materials toggle

Step 3: Algorithmic Routing Matrix

- System auto-queries available vehicles matching capacity requirements
- Displays compatible vehicles in a selection grid
- Shows estimated distance and duration based on coordinates
- Review summary and submit

6.6 Live Operations Map

Full-width Mapbox GL map showing:

- Vehicle markers (colored by status: green=Available, yellow=In-Transit, red=Maintenance)
- Route polylines for active manifests
- Side panel with manifest list for quick dispatch assignment
- Dispatch Panel: select manifest → assign driver & vehicle → confirm

6.7 Client Dashboard

Overview cards: Outstanding Deliveries, Total Freight Spent, Active Fulfillment %. Below: recent orders table with progress steppers.

6.8 Client Place Order Page

Streamlined form similar to manifest wizard but simplified for clients. Captures: origin, destination, cargo details, pickup/delivery dates. Submits to admin review queue (status=Pending).

6.9 Client Track Shipment Page

Search by trackingId. Shows ProgressStepper: Pending → Assigned → In-Transit → Delivered. Full timeline with timestamps, notes, and updatedBy info.

6.10 Client Invoices Page

Table of invoices: Invoice#, Manifest Tracking ID, Amount, Status badge, Due Date. Click row to view details.

6.11 Driver Dashboard (Mobile-First)

Today's assigned deliveries as cards. Each card shows: trackingId, origin → destination, cargo summary, scheduled pickup time. Tap to open Active Delivery.

6.12 Active Delivery Page (Mobile-First)

The core driver experience:

- **RunSheet:** Ordered checklist of delivery stops
- **StopTerminal:** Warehouse phone, gate pass, loading protocols
- **TripTimer:** Elapsed time using delta-time calculation (survives phone lock/kill)
- **Status Buttons:** Large touch targets: "Start Trip", "Arrived", "Unloading", "Complete"
- **DeliveryConfirm:** Photo capture, signature, confirmation submit

6.13 Executive Analytics Page

Dashboard of charts and widgets:

- **Fleet Utilization:** Pie chart (Available vs In-Transit vs Maintenance)
- **Route Efficiency:** Line chart (estimated vs actual distances)
- **Monthly Capacity:** Bar chart (throughput by month)
- **Delivery Performance:** Gauge (on-time % vs delayed %)
- **Revenue Summary:** Bar chart (monthly revenue) + total card

7. Real-Time Workflow (Step-by-Step)

Here's exactly how the system coordinates actions across all 4 user roles:

1. **Order Intake (Client)**
   ABC Manufacturing logs into their dashboard. They submit a bulk order: 200 televisions, 4,000 kg, Factory A to Warehouse B. Data is saved to MongoDB with status=`Pending`. The statusTimeline array gets its first entry.

2. **Administrative Control (Admin)**
   Sam the Admin checks his master console. The system reads cargo weight (4,000 kg) and auto-flags compatible trucks. Sam sees TRK-002 (Mercedes Actros, max 22,000 kg) is Available. He assigns Dave the Driver and TRK-002 with one click. Backend updates manifest: driver=Dave, vehicle=TRK-002, status=`Assigned`. Timeline entry added.

3. **Driver Execution (Driver)**
   Dave opens the app on his smartphone. His RunSheet shows the assignment with cargo details. He checks the load, boards the truck, and taps the large green "Start Trip" button. The app captures `Date.now()` as a UNIX timestamp, saves it to localStorage AND sends it to MongoDB via PATCH /api/manifests/:id/start-trip. Status changes to `In-Transit`.

4. **Real-Time Network Sync (Automatic)**
   The instant Dave's status update hits the server:
   - Sam's Admin Live Map shows Dave's truck icon change to "In-Transit" (yellow marker with route polyline)
   - ABC Manufacturing's Track Shipment page shows progress bar advance to "In-Transit" step
   - Both dashboards update via optimistic UI or polling (no manual refresh needed)

5. **Mobile Background Survival (Automatic)**
   During the 3-hour drive, Dave locks his phone, switches to GPS, and closes the browser tab. The mobile OS kills the browser process. When Dave reopens the app, the `useRecovery` hook runs on mount: it reads the saved UNIX timestamp from localStorage, computes `Date.now() - savedTimestamp = elapsed`, and the TripTimer snaps back to the correct elapsed time. No data loss.

6 **Delivery Completion (Driver)**

At Warehouse B, Dave unloads cargo, captures proof of delivery, and taps "Delivery Complete". Backend: status=`Delivered`, actualDeliveryTime=Date.now(). The vehicle is automatically set back to status=`Available`. An invoice is auto-generated for ABC Manufacturing.

7 **Invoice & Recovery (System)**

The invoice generator creates line items based on distance × client contract rate. ABC Manufacturing sees the invoice in their Invoices page. The truck returns to the available pool. All dashboards update instantly.

8. Mobile Resiliency Engineering

8.1 The Problem

When a field driver locks their smartphone during a 3-hour transit, or switches to GPS navigation, the mobile OS freezes the browser tab. If the OS needs memory, it **completely terminates** the frozen browser process. When the driver reopens the browser, the web tab forces a full reload, destroying all active JavaScript state.

8.2 Strategy A: Delta-Time Timestamp Tracking

Instead of running a vulnerable `setInterval` timer loop, the app uses time-independent mathematics:

```
// When driver taps "Start Trip":
const startTime = Date.now();
localStorage.setItem('tripStartTime', startTime);
await api.patch(`/manifests/${id}/start-trip`, { tripStartTimestamp: startTime });

// On app initialization (useRecovery hook):
useEffect(() => {
  const savedStart = localStorage.getItem('tripStartTime');
  if (savedStart) {
    const elapsed = Date.now() - parseInt(savedStart);
    setElapsedTime(elapsed);  // UI snaps to correct state
  }
}, []);
```

8.3 Strategy B: Immediate Atomic State Mutability

The mobile interface functions as a stateless controller. Every action button press sends a PATCH request to MongoDB, updating the persistent statusTimeline array. If the phone dies completely, the route state is safely stored in the cloud. Upon reboot, the app pulls the current record and builds the correct view.

8.4 Implementation Code

```
// hooks/useRecovery.js
import { useState, useEffect } from 'react';

export function useRecovery(manifestId) {
  const [elapsed, setElapsed] = useState(0);
  const [isRecovered, setIsRecovered] = useState(false);

  useEffect(() => {
```

```
    const savedStart = localStorage.getItem(`trip_${manifestId}`);
    if (savedStart) {
      const startNum = parseInt(savedStart);
      const now = Date.now();
      setElapsed(now - startNum);
      setIsRecovered(true);

      // Resume live timer from correct point
      const interval = setInterval(() => {
        setElapsed(Date.now() - startNum);
      }, 1000);

      return () => clearInterval(interval);
    }
  }, [manifestId]);

  return { elapsed, isRecovered };
}
```

8.5 Persistent Wizard State

```
// hooks/useLocalStorage.js
export function useLocalStorage(key, initialValue) {
  const [value, setValue] = useState(() => {
    const saved = localStorage.getItem(key);
    return saved ? JSON.parse(saved) : initialValue;
  });

  const updateValue = (newValue) => {
    setValue(newValue);
    localStorage.setItem(key, JSON.stringify(newValue));
  };

  const clearValue = () => {
    setValue(initialValue);
    localStorage.removeItem(key);
  };

  return [value, updateValue, clearValue];
}
```

9. MongoDB Aggregation Pipelines

9.1 Fleet Utilization Pipeline

```
Vehicle.aggregate([
  {
    $facet: {
      byStatus: [
        { $group: { _id: '$status', count: { $sum: 1 } } }
      ],
      totalWeight: [
        { $group: { _id: null, totalMaxWeight: { $sum: '$maxWeightKg' } } }
      ],
      averageEfficiency: [
        { $group: { _id: null, avgFuel: { $avg: '$fuelEfficiencyKmPerLiter' } } }
      ]
    }
  }
]);
```

9.2 Monthly Capacity Pipeline

```
Manifest.aggregate([
  {
    $match: {
      createdAt: { $gte: startDate, $lte: endDate },
      currentStatus: 'Delivered'
    }
  },
  {
    $group: {
      _id: { month: { $month: '$createdAt' }, year: { $year: '$createdAt' } },
      totalWeight: { $sum: '$cargoDetails.totalWeightKg' },
      totalVolume: { $sum: '$cargoDetails.totalVolumeCubicMeters' },
      count: { $sum: 1 }
    }
  },
  { $sort: { '_id.year': 1, '_id.month': 1 } }
]);
```

9.3 Delivery Performance Pipeline

```
Manifest.aggregate([
  {
    $match: {
      createdAt: { $gte: startDate, $lte: endDate },
      currentStatus: { $in: ['Delivered', 'Delayed'] }
```

```
    }
  },
  {
    $group: {
      _id: '$currentStatus',
      count: { $sum: 1 }
    }
  },
  {
    $group: {
      _id: null,
      total: { $sum: '$count' },
      delivered: {
        $sum: { $cond: [{ $eq: ['$_id', 'Delivered'] }, '$count', 0] }
      },
      delayed: {
        $sum: { $cond: [{ $eq: ['$_id', 'Delayed'] }, '$count', 0] }
      }
    }
  },
  {
    $project: {
      _id: 0,
      onTimeRate: { $multiply: [{ $divide: ['$delivered', '$total'] }, 100] },
      delayedRate: { $multiply: [{ $divide: ['$delayed', '$total'] }, 100] },
      total: 1
    }
  }
]);
```

9.4 Revenue Summary Pipeline

```
Invoice.aggregate([
  {
    $match: {
      createdAt: { $gte: startDate, $lte: endDate }
    }
  },
  {
    $group: {
      _id: { month: { $month: '$createdAt' }, year: { $year: '$createdAt' } },
      totalRevenue: { $sum: '$amount' },
      paidAmount: {
        $sum: { $cond: [{ $eq: ['$status', 'Paid'] }, '$amount', 0] }
      },
      pendingAmount: {
        $sum: { $cond: [{ $eq: ['$status', 'Pending'] }, '$amount', 0] }
      },
      invoiceCount: { $sum: 1 }
    }
  },
  { $sort: { '_id.year': 1, '_id.month': 1 } }
]);
```

9.5 Route Efficiency Pipeline

```
Manifest.aggregate([
  {
    $match: {
      currentStatus: 'Delivered',
      actualDeliveryTime: { $ne: null }
    }
  },
  {
    $project: {
      trackingId: 1,
      estimatedDistance: '$routing.estimatedDistanceKm',
      estimatedDuration: '$routing.estimatedDurationMinutes',
      actualDuration: {
        $divide: [
          { $subtract: ['$actualDeliveryTime', '$scheduledPickup'] },
          60000  // Convert ms to minutes
        ]
      }
    }
  }
]);
```

10. Implementation Phases (Day-by-Day)

Phase 1: Foundation (Days 1-5)

Goal: Project setup, authentication, role-based routing, all dashboard skeletons

Day	Backend Dev	Web Frontend Dev	Mobile Frontend Dev
1	Init project, install deps, setup MongoDB Atlas, create .env	Init Vite+React, install deps, setup Tailwind, create folder structure	Same as Web Dev (shared frontend setup)
2	User model, Auth routes (register/login/me), JWT middleware	AppShell layout, Sidebar, Topbar, ProtectedRoute wrapper	Redux store config, authSlice, API service setup
3	Role-based roleGate middleware, error handler, ApiError/ApiResponse utils	Login page, role-based redirect logic, toast notifications	useAuth hook, login page mobile responsiveness
4	User CRUD routes (admin), Vehicle model + routes	Admin Dashboard skeleton (stat cards + placeholder tables)	Client Dashboard skeleton, Driver Dashboard skeleton
5	Seed scripts (all users + vehicles), test all auth flows	Executive Dashboard skeleton, all page routing defined	Executive Analytics skeleton, 404 page

Deliverable: Login works for all 4 roles, each sees their own dashboard skeleton, API fully functional for auth + user management.

Phase 2: Core Data (Days 6-12)

Goal: Vehicle management, manifest CRUD, multi-step wizard, client order flow

Day	Backend Dev	Web Frontend Dev	Mobile Frontend Dev
6	Vehicle CRUD complete, /available endpoint, status management	Fleet Monitor page (DataTable with status badges, Add/Edit modal)	useLocalStorage hook, useDebounce hook
7	Manifest model (full schema with indexes), create/read/update routes	Manifest Wizard Step 1 (Partner & Node Setup form)	Shared components: StatusBadge, StatCard, ProgressStepper
8	Manifest assign route (driver + vehicle), status timeline push logic	Manifest Wizard Step 2 (Cargo Volume form)	Client Place Order form (simplified version)
9	Capacity matcher service (auto-suggest vehicles by weight/volume)	Manifest Wizard Step 3 (Route Matrix + vehicle suggestions)	Client Track Shipment page with ProgressStepper
10	Manifest list filters (by status, date range, client)	Manifest Wizard localStorage persistence + validation	Client Dashboard stats from API, order history table
11	Invoice model + generate route (auto on delivery complete)	Admin Dashboard connected to real API data	Client Invoices page
12	Invoice list/detail/pay routes, client invoice endpoints	All admin pages connected to real API, error handling	All client pages connected to real API

Deliverable: Full CRUD for vehicles and manifests, multi-step wizard working with persistence, clients can place orders and track shipments, invoices generated automatically.

Phase 3: Real-Time & Mobile (Days 13-19)

Goal: Map integration, driver mobile execution, real-time sync, optimistic UI

Day	Backend Dev	Web Frontend Dev	Mobile Frontend Dev
13	Route calculator service (Haversine distance, duration estimation)	Mapbox GL integration + vehicle markers layer	Driver Dashboard connected to API (today's assignments)
14	Notification model + routes (CRUD + unread count)	Live Map page: vehicle markers, status colors, popups	Active Delivery page: RunSheet checklist layout
15	Start-trip endpoint (capture UNIX timestamp), complete endpoint	Dispatch Panel: assign driver + vehicle UI	Stop Terminal: warehouse info, loading protocols display
16	Status update endpoint with timeline push, vehicle status auto-update	Real-time polling or WebSocket for map updates	TripTimer with delta-time recovery (useRecovery hook)
17	Driver manifest list endpoint (my assigned deliveries)	Optimistic UI mutations on manifest assignment	Status action buttons (Start Trip, Arrived, Complete)
18	Notification creation on status changes (auto-notify client/admin)	Topbar notification bell with unread count	DeliveryConfirm: photo capture, signature, submit
19	Rate limiting, input sanitization, security hardening	Full mobile responsiveness audit on admin/client pages	Mobile stress test: background/kill/recovery simulation

> **Deliverable:** Driver can execute full delivery lifecycle on mobile, admin sees live map with vehicle positions, status changes sync instantly across all dashboards, mobile state survives browser kill.

Phase 4: Analytics & Polish (Days 20-25)

Goal: Executive analytics, cron jobs, error handling, final polish

Day	Backend Dev	Web Frontend Dev	Mobile Frontend Dev
20	Fleet utilization aggregation endpoint	Fleet Utilization Chart (pie/bar)	Mobile UI polish: touch targets, contrast, spacing
21	Monthly capacity + delivery performance aggregation endpoints	Monthly Capacity + Delivery Performance charts	Loading states, empty states, error boundaries
22	Revenue summary aggregation endpoint	Revenue Summary chart + Route Efficiency chart	Toast notifications for all driver actions
23	Overdue sweep cron job (node-cron, nightly at midnight)	Executive Analytics page: all charts connected to API	Offline indicator + retry logic
24	Email service (Nodemailer) for overdue notifications	ConfirmModal for all destructive actions	Final cross-browser testing (Chrome, Safari, Firefox mobile)
25	Centralized error handler with DB rollback logic	Full app walkthrough + bug fixes	Full app walkthrough + bug fixes

Deliverable: Full executive analytics dashboard with 5 chart types, automated overdue detection and email notifications, centralized error handling, production-ready polish.

Phase 5: Testing & Deployment (Days 26-28)

Goal: Seed data, comprehensive testing, production deployment

Day	Backend Dev	Web Frontend Dev	Mobile Frontend Dev
26	Master seed script (users, vehicles, sample manifests), Postman collection	Production build testing, environment variables check	Production build testing, mobile PWA manifest
27	Deploy backend to Render/Railway, configure production env vars	Deploy frontend to Vercel/Netlify, configure API URL	Final E2E walkthrough on real mobile device
28	**Final bug fixes, README documentation, demo video recording**		

> **Deliverable:** Fully deployed, live application accessible via URL. Seed data loaded. All features working end-to-end.

11. Team Task Division

Developer 1: Backend Engineer

Area	Responsibilities
Database	All 5 Mongoose models with schemas, indexes, validation, pre-save hooks
Authentication	JWT generation/verification, bcrypt hashing, role-based middleware, token refresh
API Routes	All 30+ endpoints across 7 route files, input validation with express-validator
Controllers	All business logic: CRUD operations, status lifecycle, assignment logic
Services	Route calculator (Haversine), capacity matcher, invoice generator, email service
Aggregation	MongoDB $lookup, $match, $facet pipelines for all 5 analytics endpoints
Cron Jobs	Node-cron overdue sweep, automated email notifications
Error Handling	Centralized error handler, ApiError class, DB transaction rollback
Seeds	Sample data for all models (users, vehicles, manifests)
Deployment	Render/Railway deployment, environment configuration, MongoDB Atlas setup

Developer 2: Web Frontend Engineer (Admin + Executive)

Area	Responsibilities
Layout	AppShell, Sidebar (role-based nav), Topbar (notifications, logout), ProtectedRoute
Login	Login page, JWT storage, role-based redirect after auth
Admin Dashboard	Stat cards, recent manifests table, quick actions
Fleet Monitor	Vehicle data grid, status badges, Add/Edit/Delete vehicle modal
Manifest Wizard	3-step form (Partner, Cargo, Route), localStorage persistence, validation
Live Map	Mapbox GL integration, vehicle markers, route polylines, dispatch panel
Executive Analytics	5 chart components using Recharts, KPI widgets, data aggregation display
Client Invoices	Invoice list table, status badges, detail view

State Management	Redux store, authSlice, manifestSlice, vehicleSlice, uiSlice
API Layer	All API service files, axios interceptors, error handling
Deployment	Vercel/Netlify deployment, production environment variables

Developer 3: Mobile Frontend Engineer (Client + Driver)

Area	Responsibilities
Client Dashboard	Outstanding deliveries, spending overview, fulfillment percentages
Place Order	Simplified bulk freight request form, date pickers, validation
Track Shipment	ProgressStepper component, timeline display, trackingId search
Client Invoices	Mobile-optimized invoice list, status badges
Driver Dashboard	Today's delivery cards, quick stats, mobile-first layout
Active Delivery	RunSheet checklist, StopTerminal, DeliveryConfirm with photo/signature
TripTimer	Delta-time calculation using UNIX timestamps, survives background/kill
Recovery Hooks	useRecovery (trip state), useLocalStorage (form persistence)
Mobile Polish	Touch targets (min 44px), high contrast, viewport optimization, PWA manifest
Shared Components	StatusBadge, StatCard, ProgressStepper, LoadingSpinner, EmptyState
Testing	Cross-browser mobile testing, background/kill recovery simulation

12. Seed Data

12.1 User Accounts

Name	Email	Password	Role	Extra Fields
Sam Manager	admin@logistics.com	admin123	admin	—
Sarah Director	exec@logistics.com	exec123	executive	—
ABC Manufacturing	client@abc.com	client123	client	company: ABC Manufacturing, contractRate: 2.5
XYZ Wholesale	client@xyz.com	client123	client	company: XYZ Wholesale, contractRate: 3.0
Dave Johnson	driver1@logistics.com	driver123	driver	licenseNumber: DL-9876
Mike Smith	driver2@logistics.com	driver123	driver	licenseNumber: DL-5432

12.2 Vehicle Fleet

Registration	Model	Make	Year	Max Weight (kg)	Max Volume (m^3)	Fuel Efficiency (km/L)
TRK-001	FH16	Volvo	2022	18,000	60	3.5
TRK-002	Actros	Mercedes	2023	22,000	75	3.2
TRK-003	Stralis	Iveco	2021	15,000	45	4.0
TRK-004	TGA	MAN	2023	20,000	65	3.8
TRK-005	XF	DAF	2022	16,000	50	3.6

12.3 Sample Manifests

Client	Cargo	Weight	Origin	Destination	Status
ABC Manufacturing	200 TVs	4,000 kg	Factory A, Chicago	Warehouse B, Detroit	Pending
XYZ Wholesale	500 Boxes Electronics	8,500 kg	Port Newark, NJ	Distribution Center, PA	Assigned

ABC Manufacturing	Industrial Parts	12,000 kg	Plant C, Ohio	Assembly Line D, MI	In-Transit
XYZ Wholesale	Furniture Set	3,200 kg	Warehouse E, NY	Retail Store F, CT	Delivered

13. Environment Configuration

Backend .env

```
PORT=5000
MONGODB_URI=mongodb+srv://username:password@cluster0.xxxxx.mongodb.net/b2b-logistics?
retryWrites=true&w=majority
JWT_SECRET=b2b-logistics-super-secret-key-change-in-production-2026
JWT_EXPIRES_IN=7d
NODE_ENV=development

# Email (Nodemailer)
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=your-email@gmail.com
SMTP_PASS=your-app-password-here

# Frontend URL (CORS)
CLIENT_URL=http://localhost:5173
```

Frontend .env

```
VITE_API_URL=http://localhost:5000/api
VITE_MAPBOX_TOKEN=pk.eyJ1IjoibWFwYm94IiwiYSI6ImNpejY4NXVycTA2emYycXBndHRqcmZ3N3gifQ.rJcFIG2
```

14. Setup Commands

```
# ===== INITIALIZE PROJECT =====
mkdir b2b-logistics && cd b2b-logistics
git init

# ===== BACKEND SETUP =====
mkdir backend && cd backend
npm init -y

# Core dependencies
npm install express mongoose jsonwebtoken bcryptjs cors dotenv express-validator

# Utilities
npm install node-cron nodemailer multer

# Dev dependencies
npm install -D nodemon prettier

# Create folder structure
mkdir config middleware models routes controllers services cron utils seeds

cd ..

# ===== FRONTEND SETUP =====
npm create vite@latest frontend -- --template react
cd frontend

# Core dependencies
npm install react-router-dom @reduxjs/toolkit @tanstack/react-query axios

# UI dependencies
npm install @mapbox/mapbox-gl recharts react-hot-toast

# Dev dependencies
npm install -D tailwindcss @tailwindcss/vite

# Create folder structure
mkdir -p src/{store,services,hooks,utils,styles}
mkdir -p src/components/{layout,shared,admin/ManifestWizard,client,driver,executive}
mkdir -p src/pages/{admin,client,driver,executive}

cd ..

# ===== ROOT SETUP =====
# Create root package.json for concurrently
npm init -y
npm install -D concurrently

# ===== CONNECT TO GITHUB =====
git remote add origin https://github.com/your-username/b2b-logistics.git
git add .
git commit -m "Initial project setup - B2B Logistics Platform"
```

```
git branch -M main
git push -u origin main
```

Development Start Commands

```
# Backend (from /backend)
npx nodemon server.js

# Frontend (from /frontend)
npm run dev

# Or both at once (from root, if using concurrently)
npm run dev   # "concurrently \"cd backend && nodemon server.js\" \"cd frontend && npm
run dev\""
```

15. Engineering Interview Talking Points

> **For Technical Recruiters:** These are the key architectural decisions and engineering patterns you can discuss when presenting this project.

15.1 Why MongoDB Over SQL?

The Manifest model uses deeply nested sub-documents (cargoDetails, routing, statusTimeline) that map naturally to JSON. SQL would require 4+ junction tables to achieve the same structure. MongoDB's aggregation framework ($lookup, $facet, $match) handles the analytics queries server-side, reducing payload sizes by 80%+.

15.2 Server-Side Aggregation vs Client-Side Loops

Fleet utilization metrics are computed entirely in MongoDB pipelines using $facet to run parallel computations. This avoids sending raw data to the client and looping in JavaScript, which would cause severe lag with large datasets.

15.3 Optimistic UI Pattern

When a driver taps "Complete Delivery," the UI immediately shows the success state while the server request runs in the background. If the request fails, the UI rolls back. This hides network latency and makes the app feel instant.

15.4 Mobile State Resilience

Instead of storing elapsed time in React state (volatile), we store a UNIX timestamp in localStorage and MongoDB. On recovery, we compute elapsed time mathematically: `now - savedTimestamp`.

This is time-independent and survives any browser kill scenario.

15.5 Role-Based Architecture

One backend, one database, four user interfaces. The JWT token contains role claims. The React router reads these claims and conditionally renders navigation items and page access. No separate apps needed.

15.6 Centralized Error Handling

All errors bubble up to a single Express error handler middleware. Database transaction rollbacks ensure data consistency. The ApiError class standardizes error responses across all endpoints.

15.7 Automated Background Jobs

Node-cron runs a nightly sweep to detect overdue deliveries, update their status to "Delayed," and trigger email notifications via Nodemailer. This demonstrates server-side automation without external services.
